

Supervision

Student

Supervisor

Posted at 04/09/2026

Main activities	During this month, the main focus of the internship was the implementation of the AWS migration for Outsight Cloud. The objective of this work was to design a first cloud-native deployment architecture capable of replacing the current Docker/Traefik-based hosting approach with a more scalable and maintainable infrastructure on AWS. The first phase consisted of defining the target infrastructure and translating it into Infrastructure as Code using Terraform. A complete Terraform stack was prepared to provision the main AWS components required to run Outsight Cloud, including: - an Application Load Balancer (ALB) to expose the platform publicly - ECS Fargate services to run the frontend and backend containers - Route 53 and ACM integration to manage DNS and HTTPS certificates - CloudWatch log groups for centralized application logs - S3 and CloudFront for simulation file storage and delivery Several architectural choices were made during this phase. First, ECS Fargate was selected as the compute platform in order to avoid direct server management and simplify deployment operations. This choice allows the application containers to run in a managed environment while keeping the deployment model close to the existing containerized architecture. Second, the routing logic was redesigned so that the frontend is served from the root path while the backend API is exposed under /api through the ALB. To support this change, the Express backend was updated so that API routes can be mounted under /api, while preserving the /status endpoint for compatibility with existing health checks. Third, simulation file storage was redesigned around a private S3 bucket fronted by CloudFront. This choice was made to improve scalability and reliability while keeping the storage bucket non-public. Access to the bucket is restricted through CloudFront origin access control, and Terraform also defines IAM permissions for upload workflows. Another important activity was the preparation of a secure container image delivery workflow. Since the project currently uses images stored in GitLab Container Registry, AWS Secrets Manager was integrated so ECS tasks can retrieve registry credentials securely without storing passwords directly in Terraform state. Finally, the migration work also included adapting the application configuration for the future AWS domain structure.
Achievement(s)	Several deliverables were produced during this period. First, a complete Terraform-based AWS deployment foundation was created for Outsight Cloud.

This infrastructure definition now covers the main components needed to host the application on AWS, including networking exposure, container orchestration, DNS, HTTPS, logging, and storage.

Second, the future deployment architecture was clarified and documented through code. The migration now follows a clear structure in which:

the frontend and backend run as separate ECS services
 the ALB routes / to the frontend and /api to the backend
 HTTPS can be enabled through ACM certificates
 public DNS can be managed either through an existing Route 53 zone or through a delegated subdomain

Third, a new storage model for simulation files was prepared.

A private S3 bucket and a CloudFront distribution were configured to provide secure and scalable file delivery. In addition, IAM resources were created for CI-based upload workflows, including a dedicated user, an assumable role, and permissions for S3 object management and CloudFront cache invalidation.

Another significant achievement was improving the backend compatibility with the new infrastructure.

The API routing was updated so that the application can operate correctly behind the AWS load balancer without relying on path rewriting at the proxy level. This is an important step for simplifying the deployment architecture and making the routing behavior more explicit.

Overall, this month established the first concrete implementation of the AWS migration and transformed the migration effort from an architectural idea into an operational deployment foundation.

Troubleshooting(s)

One major difficulty was translating the current platform behavior into an AWS-compatible routing model.

The existing setup relied on reverse proxy behavior that stripped the /api prefix before forwarding requests to the backend. In the AWS architecture, the backend had to be adapted so it could natively support the /api path while preserving compatibility for health checks and existing behavior.

A further difficulty concerned secure access to private container images.

Since images are pulled from GitLab Container Registry, it was necessary to design a solution that would allow ECS tasks to authenticate securely without exposing credentials in Terraform code or state files. This led to the integration of AWS Secrets Manager and dedicated IAM permissions for the ECS execution role.

Finally, designing the storage migration for simulation assets also required careful consideration. The objective was not only to move files to AWS, but also to ensure that storage remains private, publicly accessible only through a controlled CDN layer, and compatible with future CI/CD upload workflows.

Next step(s)

The next phase of the internship will focus on completing the AWS sandbox deployment and starting the replacement of Airtable with PostgreSQL hosted on Amazon Aurora.

A first objective will be to validate the new AWS environment by testing ECS services, ALB routing, HTTPS, DNS configuration, logging, and the new S3/CloudFront storage flow for

simulation files.

In parallel, work will begin on the database migration. This will include designing the first Aurora PostgreSQL schema for the main Oversight Cloud entities and progressively adapting the backend API so that the endpoints currently connected to Airtable can interact with the new database instead.

Finally, the next steps will also include defining a progressive migration strategy, so that Airtable can be replaced step by step while keeping the platform operational and preparing a more scalable AWS-native architecture.

**Intermediate report
(only one per month)**
